Case Study Report

on

Automated Build Pipeline for Web Applications using Jenkins

Bachelor of Technology
in
Computer Science Engineering (AI & ML)

By

D Kiranmai Meghana	A23126552141
G Padma Sree	A23126552144
M Sahiti	A23126552163
V Anjali Devi	A23126552185
Y Ganga Yamuna	A23126552189

III/IV CSE(AI&ML) – II Semester

PRAGNANAM BRAHMA
ANITS

CERTIFICATE

The case study report entitled "**Automated build Pipeline for Web Applications using Jenkins**" submitted by D Kiranmai Meghana(A23126552141), G Padma Sree(A23126552144), M Sahiti(A23126552163), V Anjali Devi(A23126552185), Y Ganga Yamuna (A23126552189), III/IV B.Tech, II-Semester, Dept. of CSE (AI &ML) is found to be satisfactory.

Mr. P. Santosh Kumar
Assistant Professor
Department of CSE(AI&ML)

ACKNOWLEDGEMENT

This case study reflects the students for part of their academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world environment. The student expresses sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering (AI & ML) at ANITS, and Mr P. Santhosh for their invaluable guidance, and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support in this case study.

D Kiranmai Meghana
Regd. No. A23126552141
G Padma Sree
Regd.No. A23126552144
M Sahiti
Regd.No A23126552163
V Anjali Devi
Regd.No A23126552185
Y Ganga Yamuna
Regd.No A23126552189

ABSTRACT

In modern software development, automation plays a key role in improving efficiency and reducing errors. This case study focuses on the implementation of an automated build pipeline for a web application using Jenkins.

The main objective of this study is to design a Continuous Integration (CI) pipeline that automates the process of code integration, building, testing, and deployment. By integrating tools such as Git, GitHub, and Jenkins, the system ensures that the latest code changes are automatically processed and deployed without manual intervention.

The pipeline is deployed on a cloud-based environment using AWS EC2, where the application is hosted and executed. Various stages such as code cloning, building, testing, and deployment are performed sequentially to ensure smooth operation and reliability.

This automation reduces deployment time, minimizes human errors, and improves overall software quality. The case study demonstrates how DevOps practices can be effectively used to achieve faster and more reliable application delivery.

Overall, the implementation of the automated build pipeline highlights the importance of CI/CD in modern development environments and provides a scalable solution for efficient software deployment.

TABLE OF CONTENTS

1. Introduction 1
2. Objectives 1
3. System Architecture (AWS) 2
   - 3.1 Infrastructure Setup (Instance Configuration) 2
   - 3.2 Network Configuration (Security & Firewalls) 2
4. Tools and Technologies Used 3
   - 4.1 Software Stack 3
   - 4.2 Permission Management 3
5. Methodology / Implementation 5
   - 5.1 Initializing the Pipeline Project 5
   - 5.2 Jenkins Configuration (Integrated Pipeline Scripting) 6
   - 5.3 Pipeline Stages (Stage-by-Stage Breakdown) 8
   - 5.4 Automated Execution & Monitoring 8
   - 5.5 Webhook-Driven Automation 8
   - 5.6 Execution & Monitoring 9
   - 5.7 Benefits of Automation 9
6. Challenges & Optimization 9
   - 6.1 Resource Constraint Resolution 9
7. Verification & Results 9
8. Conclusion 10
9. Future Scope 10

1. Introduction

In today's fast-growing software industry, the demand for faster and more reliable application delivery has increased significantly. Traditional methods of software development and deployment involve manual processes, which are time-consuming and prone to errors. To overcome these challenges, DevOps practices have been introduced to automate and streamline the development lifecycle.

DevOps is a combination of development and operations that focuses on continuous integration and continuous deployment (CI/CD). It enables teams to build, test, and deploy applications quickly and efficiently. One of the most widely used tools for implementing CI/CD pipelines is Jenkins.

This case study focuses on the development of an automated build pipeline for a web application using Jenkins. The pipeline is designed to automatically fetch the latest code from a version control system, build the application, run tests, and deploy it to a server without manual intervention.

The entire setup is implemented using cloud infrastructure, specifically AWS EC2, which provides a scalable and flexible environment for hosting both the automation server and the application. By integrating tools such as Git, GitHub, and Jenkins, the system ensures a smooth and continuous workflow.

This approach helps in reducing human errors, saving time, and improving the overall quality of software delivery. The case study demonstrates how automation can transform the traditional development process into a more efficient and reliable system.

2. Objectives

The main objectives of this case study are:

- To understand the concept of DevOps and Continuous Integration/Continuous Deployment (CI/CD).
- To design and implement an automated build pipeline for a web application using Jenkins.
- To automate the process of code integration, building, testing, and deployment.
- To reduce manual effort and minimize human errors in the deployment process.
- To improve the speed and efficiency of software delivery.
- To deploy the application on a cloud platform using AWS EC2.
- To ensure continuous monitoring and smooth functioning of the application

3.System Architecture (AWS)

To host the automation server and the live application, an **Amazon EC2** instance was provisioned with a focus on cost-efficiency and security.

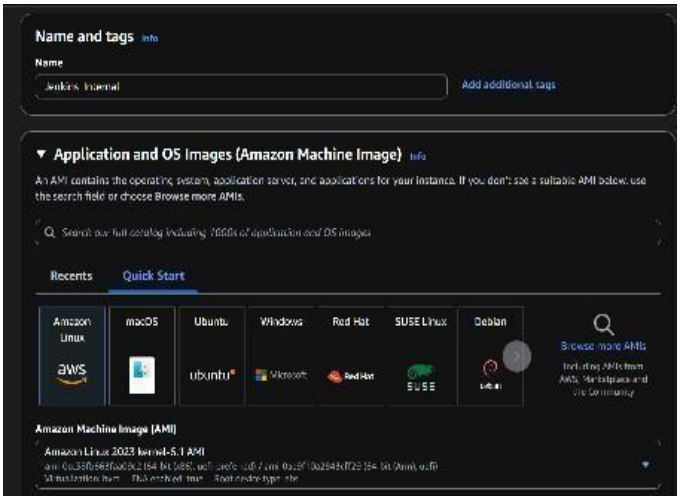
3.1 Infrastructure Setup (Instance Configuration)

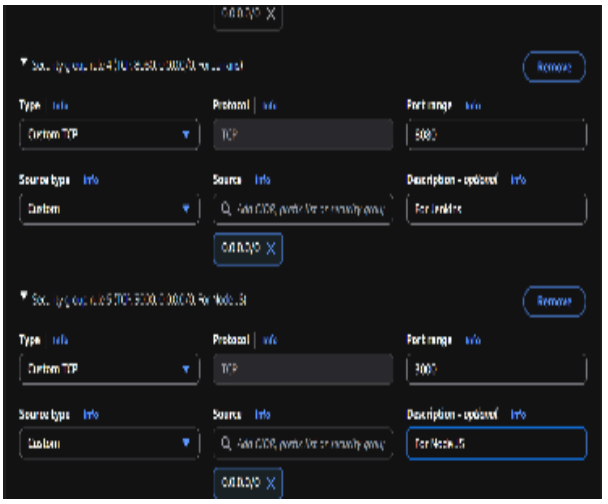
- **AMI:** Amazon Linux 2023
- **Instance Type:** t3.micro (Free Tier)
- **Storage:** 8 GiB EBS Volume

3.2 Network Configuration (Security & Firewalls)

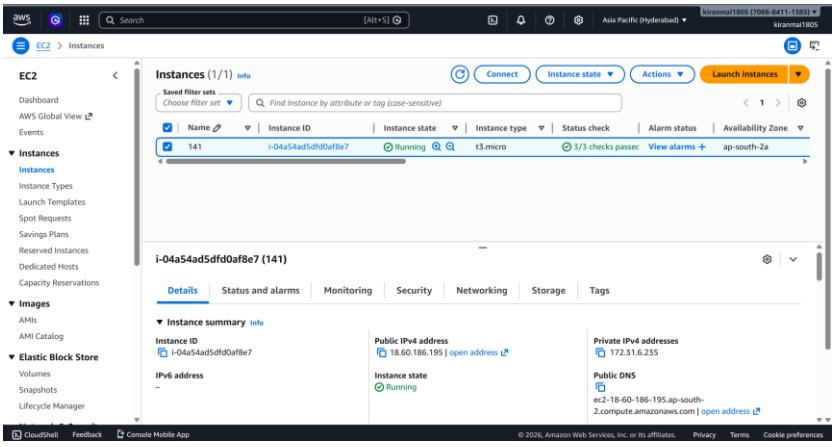
The AWS Security Group was hardened with specific inbound rules to ensure safe traffic flow:

- **SSH (Port 22):** Restricted access for administrative terminal management.
- **Jenkins (Port 8080):** Public access for the CI/CD Dashboard.





Figs: Instance creation with allowing certain ports (8080)



4. Tools and Technologies Used (Environment Provisioning & Tooling)

Once the instance was live, the environment was configured via SSH. The following software stack was installed to support the automation lifecycle:

4.1 Software Stack

- **Java 17 (Amazon Corretto):** The core runtime required for Jenkins.
- **Jenkins v2.4x:** The central orchestration engine.
- **Git:** For source code version control.

4.2 Permission Management

To allow Jenkins to manage system processes (such as killing old application instances on Port 3000), the jenkins user was granted password-less sudo
privileges within the /etc/sudoers configuration.

```
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kiran>cd Downloads

C:\Users\kiran\Downloads>ssh -i "Jenkins_inter.pem" ec2-user@13.51.162.60
The authenticity of host '13.51.162.60 (13.51.162.60)' can't be established.
ED25519 key fingerprint is SHA256:rqpMiUkv2pypr0U9Li7WYi2iz5ORAfP11daRHqHJcRI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.51.162.60' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-172-31-47-144 ~]$
```

Fig: Connecting to ec2 using ssh

```
[ec2-user@ip-172-31-47-144 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository                    265 kB/s |  31 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-47-144 ~]$
```

Fig: Updating packages in the server

```
[ec2-user@ip-172-31-47-144 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo \
    https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2026-03-29 10:33:24--  https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 199.232.174.133, 2a04:4e42:6b::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|199.232.174.133|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://pkg.jenkins.io/rpm-stable/jenkins.repo [following]
--2026-03-29 10:33:24--  https://pkg.jenkins.io/rpm-stable/jenkins.repo
Reusing existing connection to pkg.jenkins.io:443.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo        100%[===================================================================>]     267  --.-KB/s    in 0s

2026-03-29 10:33:24 (13.1 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[ec2-user@ip-172-31-47-144 ~]$
```

```
[ec2-user@ip-172-31-47-144 ~]$ sudo dnf install git -y
Last metadata expiration check: 0:36:14 ago on Sun Mar 29 10:34:12 2026.
Dependencies resolved.
================================================================================================================
 Package                     Architecture      Version                              Repository              Size
================================================================================================================
Installing:
 git                         x86_64            2.50.1-1.amzn2023.0.1                amazonlinux             53 k
Installing dependencies:
 git-core                    x86_64            2.50.1-1.amzn2023.0.1                amazonlinux            4.9 M
 git-core-doc                noarch            2.50.1-1.amzn2023.0.1                amazonlinux            2.8 M
 perl-Error                  noarch            1:0.17030-2.amzn2023.0.1             amazonlinux             42 k
 perl-File-Find              noarch            1.37-477.amzn2023.0.7                amazonlinux             25 k
 perl-Git                    noarch            2.50.1-1.amzn2023.0.1                amazonlinux             41 k
 perl-TermReadKey            x86_64            2.38-9.amzn2023.0.2                  amazonlinux             36 k
 perl-lib                    x86_64            0.65-477.amzn2023.0.7                amazonlinux             15 k
```

Fig: Installing Git

```
[ec2-user@ip-172-31-47-144 ~]$ sudo dnf install jenkins -y
Last metadata expiration check: 0:00:52 ago on Sun Mar 29 10:34:12 2026.
Dependencies resolved.
================================================================================================================
 Package                     Architecture                     Version                          Repository                 Size
================================================================================================================
Installing:
 jenkins                     noarch                           2.541.3-1                        jenkins                    92 M
```

Fig: Installing Jenkins

```
[ec2-user@ip-172-31-47-144 ~]$ sudo systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
[ec2-user@ip-172-31-47-144 ~]$ sudo systemctl start jenkins
[ec2-user@ip-172-31-47-144 ~]$
```

Fig: Enabling Jenkins

5. Methodology (Pipeline Design & Automation Strategy)

The core of this project is the Pipeline as Code implementation. This section details the step-by-step configuration within the Jenkins UI to establish a

seamless connection between the source code and the deployment environment.

5.1 Initializing the Pipeline Project

We began by creating a new Pipeline Item named Web-Build_pipeline. Unlike a Freestyle project, the Pipeline project allows for a more flexible, script-based workflow that can handle complex deployment stages.

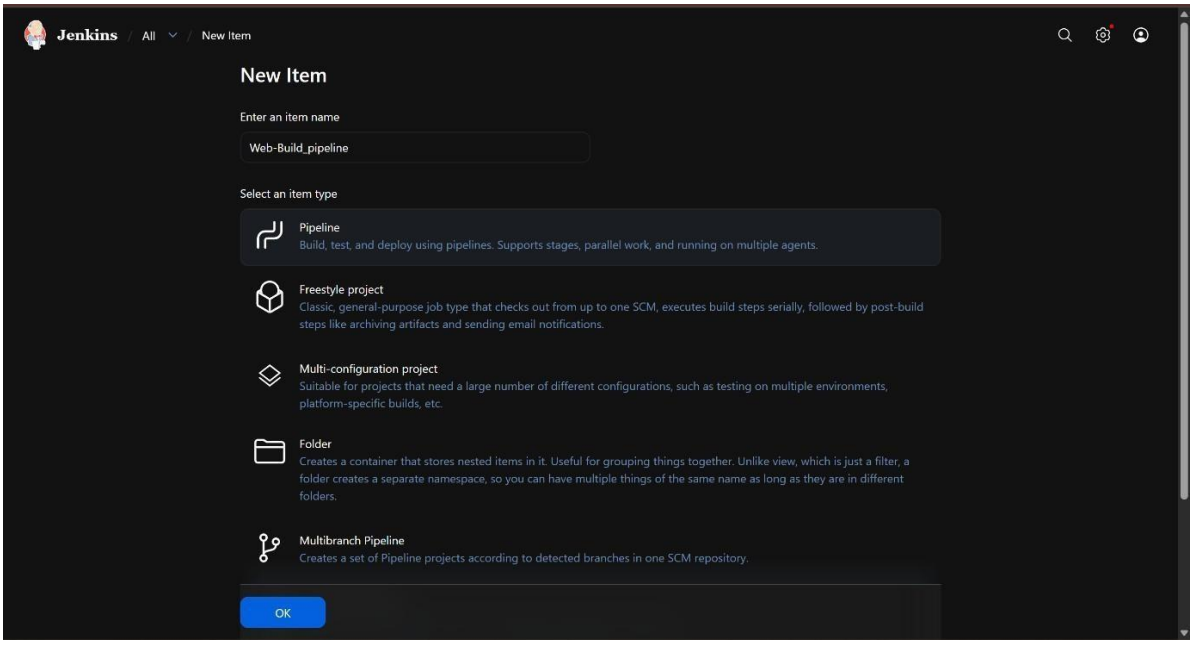


Fig: Selecting pipeline

5.2 Jenkins Configuration (Integrated Pipeline Scripting)

The pipeline is governed by a Declarative Script. This script defines the specific environment (Agent) and the sequence of actions (Stages) required to transform raw code into a running application.

Key Configuration Details:

- Agent Any: Instructs Jenkins to run the build on the available "Built-In" node.
- Git Integration: We linked the pipeline to the GitHub repository URL. This ensures that Jenkins has the correct "Source of Truth" for the application code.

Pipeline Script

```
pipeline {
    agent any

    stages {
        stage('Clone Repository') {
            steps {
                // Clones your Robot repository
                git branch: 'main', url: 'https://github.com/<github_username>/ROBOT.git'
            }
        }

        stage('Deploy to Web Server') {
            steps {
                echo 'Deploying static files to Apache...'
                // Copies the index.html to the Apache web root
                sh 'cp index.html /var/www/html/index.html'
            }
        }
    }
}
```

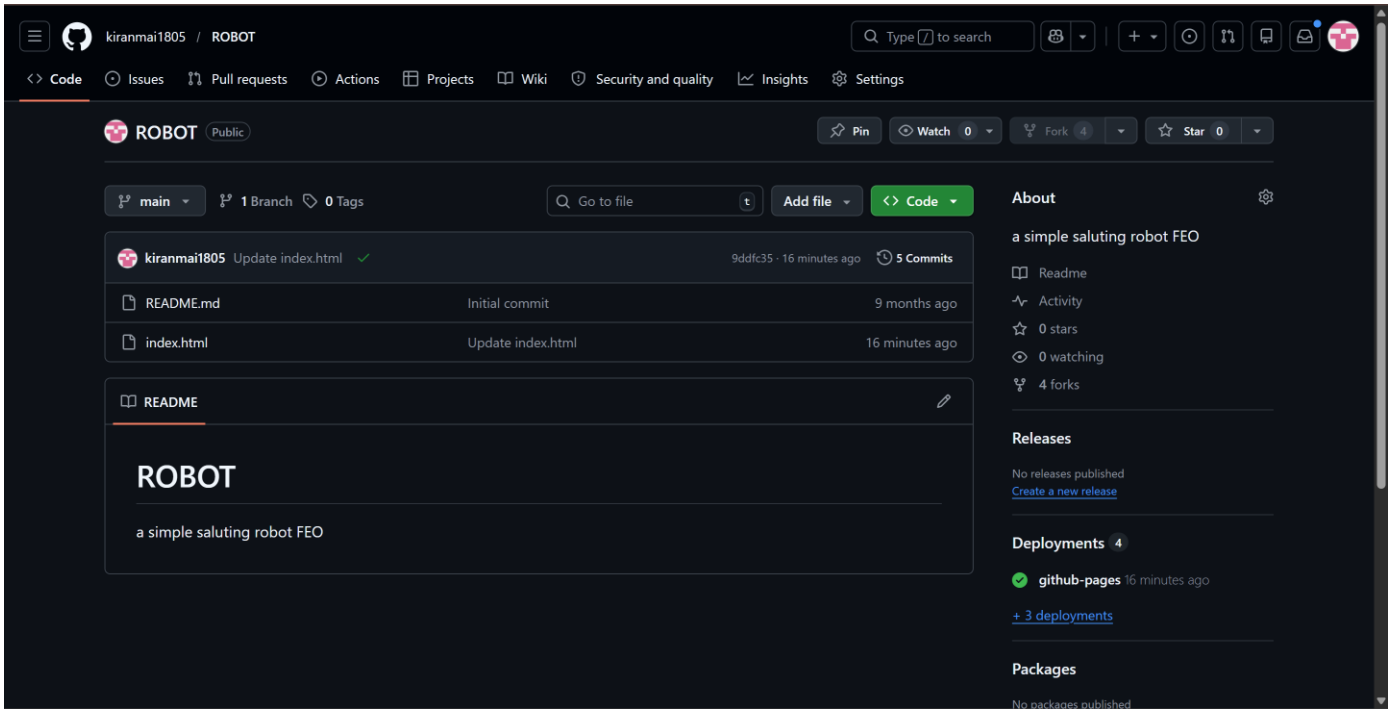
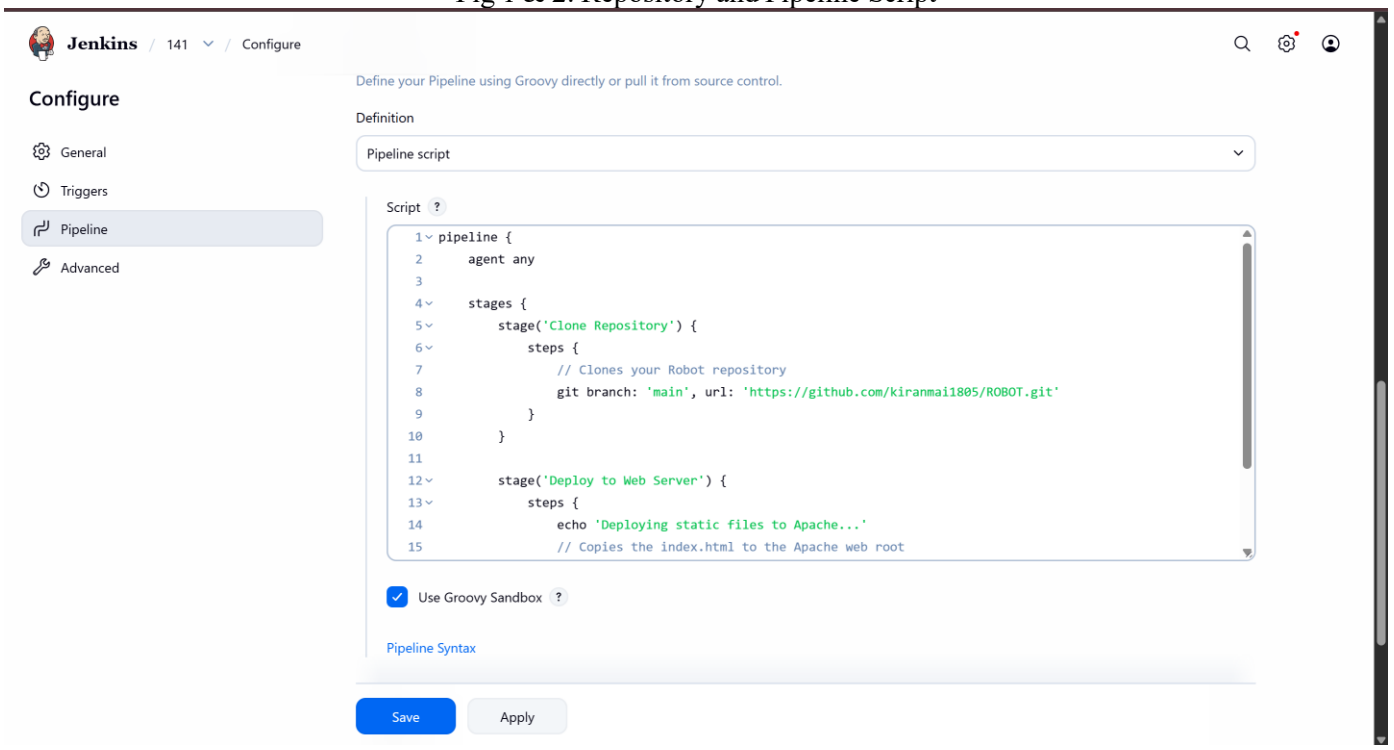


Fig 1 & 2: Repository and Pipeline Script



5.3 Pipeline Stages (Stage-by-Stage Breakdown)

To ensure the highest reliability, the pipeline was divided into four distinct logical blocks:

1. **Stage: Clone Code** Jenkins initializes the workspace and pulls the latest commit from the main branch. This ensures we are always deploying the most recent version of the software.

2. **Stage: Build** In this stage, the system prepares the environment for deployment. While static applications do not require external library installations (like npm install), this phase remains critical for verifying the source code's integrity and ensuring the workspace is clean before moving to the production environment.

3. **Stage: Deploy** This stage automates the transfer of the application to the live environment. It replaces the manual process of moving files by copying the index.html directly into the Apache web root directory (/var/www/html). This ensures the latest version of the "Robot Salute" page is immediately accessible via the Public IP on Port 80 without requiring a manual server restart or persistent terminal connection.

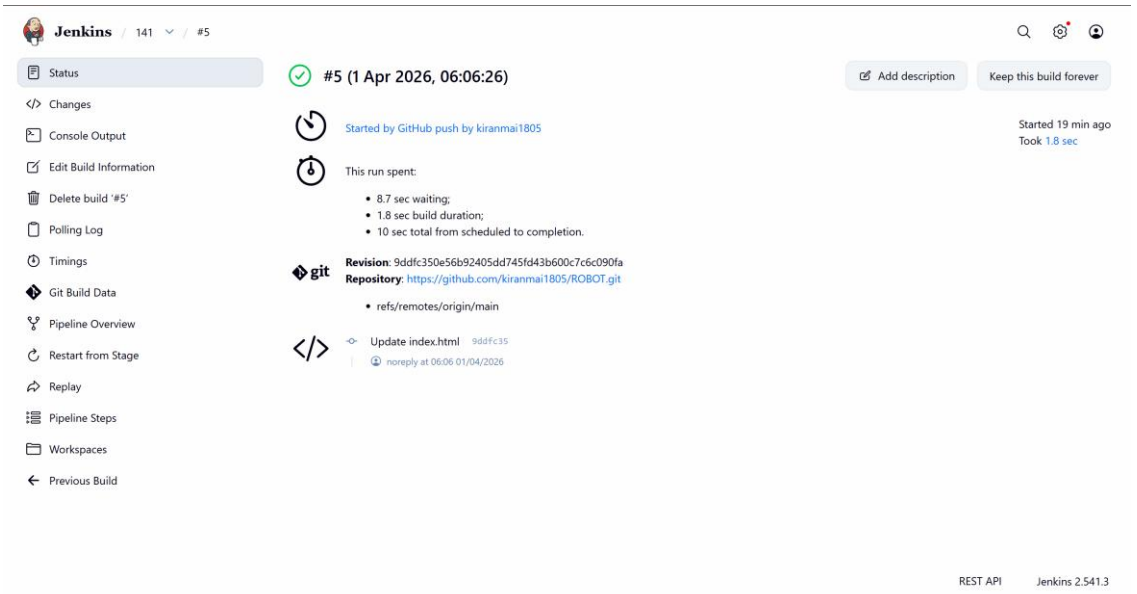


Fig: Output Console showing Build Success

5.4 Automated Execution & Monitoring

Once the design was finalized, we triggered the Build Now command. Jenkins provides a visual "Stage View" that tracks the progress of each phase in real-time, providing immediate feedback on the health of the deployment.

5.5 Webhook-Driven Automation

Instead of manual intervention, the pipeline uses GitHub Webhooks to create an event-driven system. When code is pushed to the ROBOT repository, GitHub sends a real-time "ping" to Jenkins, triggering the build automatically.

5.6 Execution & Monitoring

Once triggered, Jenkins executes the Clone, Test, and Deploy stages sequentially. The Stage View dashboard provides real-time visual feedback on the health of each phase.

- **Continuous Integration (CI):** Jenkins automatically fetches the latest code from the version control system to ensure the workspace is up to date.
- **Continuous Deployment (CD):** The script automatically overwrites the index.html in the Apache web root (/var/www/html), reflecting changes live at the Public IP without manual intervention

5.7 Benefits of Automation

- **Efficiency:** Significantly reduces deployment time and eliminates manual process errors.
- **Reliability:** Ensures a smooth and continuous workflow, improving the overall quality of software delivery.

6. Challenges & Optimization (Case-Specific)

6.1 Resource Constraint Resolution

During the implementation on a t3.micro instance, we encountered a "Node Offline" issue due to Jenkins' default high disk-space thresholds.

- Action taken: We manually reconfigured the Free Space Threshold to 200MiB within the Node Configuration. This allowed the automation to run smoothly on limited-tier hardware without service interruptions.

7. Verification & Results

The success of the project was confirmed through the following deliverables:

- **Pipeline Success:** The Jenkins dashboard displayed a 100% success rate for all stages (Clone, Build, Deploy).
- **Live Deployment:** The application was successfully reached at the EC2 Public IP confirming that the "ROBOT" app was active and listening.
- **Process Automation:** The fuser logic ensured that redeployments were seamless with zero manual intervention required to clear ports.

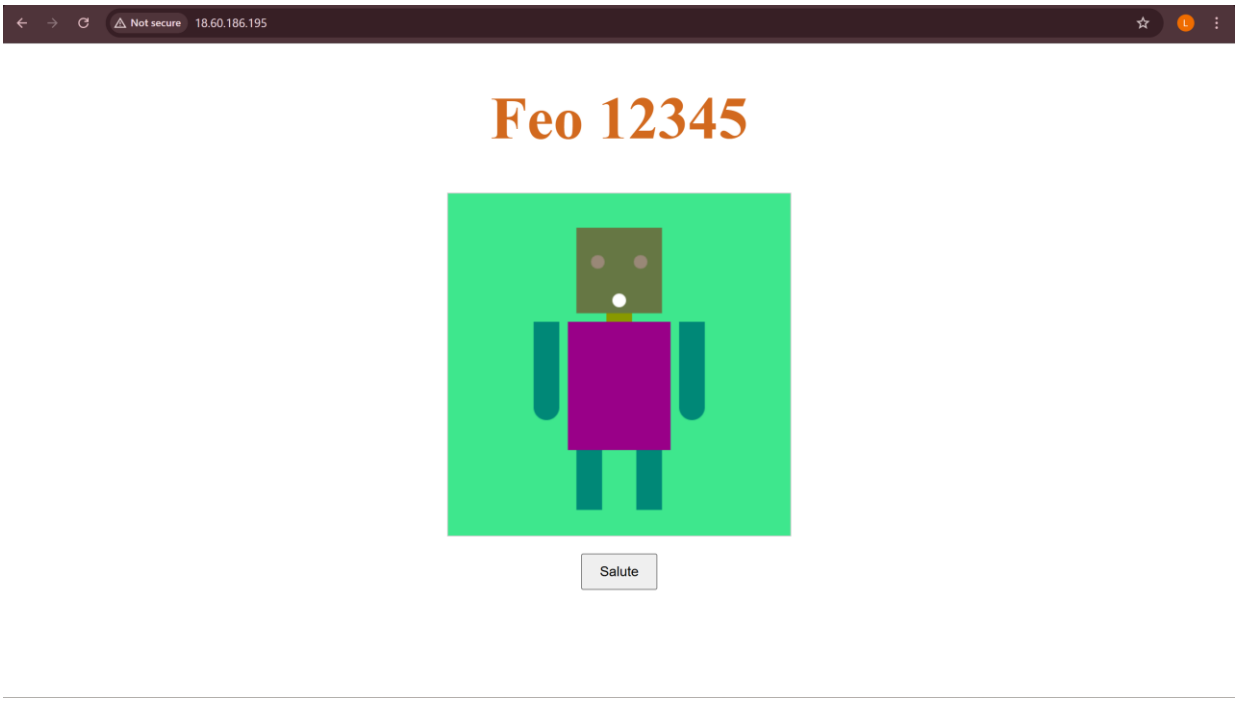


Fig: Simple Robot application running on the ec2 Server

8. Conclusion

This project successfully demonstrates the power of DevOps automation. By integrating AWS, Jenkins, and GitHub, we created a robust framework that allows developers to focus on code, while the infrastructure handles the delivery. The result is a more stable, scalable, and faster deployment process.

9. Future Scope

The current system can be further improved by implementing advanced tools and technologies in the DevOps process. Containerization can be introduced using Docker to make the application more portable and consistent across different environments.

Kubernetes can be used for container orchestration to enable better scalability and management of applications. Monitoring tools such as Prometheus and Grafana can be integrated to track system performance and detect issues in real-time.

Additionally, security can be enhanced by implementing proper authentication and access control mechanisms within the CI/CD pipeline. The system can also be extended to support multiple environments such as development, testing, and production.

These improvements will make the system more efficient, scalable, and reliable for real-world applications.